SwiftData

Material complementar sobre SwiftData para o treinamento de iOS da Capgemini e Ready Proway.

Vamos usar como base o modelo:

```
// Importações
import Foundation
import SwiftData

// Classe
@Model
class Tarefa {
    var titulo: String
    var descricao: String
    var data: Date

    init(titulo: String, descricao: String, data: Date = .now) {
        self.titulo = titulo
        self.descricao = descricao
        self.data = data
    }
}
```

Implementar as funções para: cadastrar, selecionar, alterar e remover:

Listar todas as tarefas

```
func listarTarefas() {
        let descriptor = FetchDescriptor<Tarefa>()
        tarefas = (try? modelContext.fetch(descriptor)) ?? []
}
```

Cadastrar uma nova tarefa

```
func cadastrarTarefa() {
        let objTarefa = Tarefa(titulo: titulo, descricao: descricao, data: data)
        modelContext.insert(objTarefa)
}
```

Atualizar uma tarefa existente

```
// Preparar tarefa para edição (Carrega os dados nos campos)
func selecionarTarefaParaEdicao(_ tarefa: Tarefa) {
        tarefaSelecionada = tarefa
        titulo = tarefa.titulo
        descricao = tarefa.descricao
        data = tarefa.data
}

func atualizarTarefa() {
        guard let tarefa = tarefaSelecionada else { return }

        // Alterar propriedades diretamente (O SwiftData rastreia e salva automaticamente)
        tarefa.titulo = titulo
        tarefa.descricao = descricao
        tarefa.data = data
}
```

Remover uma tarefa

```
func removerTarefa(offSets: IndexSet) {
    guard let indice = offSets.first else { return }
    modelContext.delete(tarefas[indice])
}
```

Vamos aprender a implementar nossas consultas, veja abaixo alguns exemplos:

1. ORDER BY (SELECT * FROM tarefa ORDER BY titulo ASC / data DESC)

```
func listarTarefasOrdenadas() {
    // Define os critérios de ordenação
    let ordenacao = [
        SortDescriptor(\Tarefa.titulo, order: .forward), // Crescente (A-Z)
            SortDescriptor(\Tarefa.data, order: .reverse)      // Decrescente (Mais recente primeiro)
    ]

    let descriptor = FetchDescriptor<Tarefa>(sortBy: ordenacao)
    tarefas = (try? modelContext.fetch(descriptor)) ?? []
}
```

2. WHERE Exato (SELECT * FROM tarefa WHERE titulo = ?)

```
func buscarPorTituloExato(termo: String) {
    // A macro #Predicate cria o filtro SQL de forma segura
    let filtro = #Predicate<Tarefa> { tarefa in
        tarefa.titulo == termo
    }

    let descriptor = FetchDescriptor<Tarefa>(predicate: filtro)
    tarefas = (try? modelContext.fetch(descriptor)) ?? []
}
```

3. LIKE / Contém texto (SELECT * FROM tarefa WHERE titulo LIKE '%termo%')

```
func pesquisarTarefas(textoBusca: String) {
    // Se a busca estiver vazia, retorna todas as tarefas
    guard !textoBusca.isEmpty else {
        listarTarefas()
        return
    }

     // localizedStandardContains ignora maiúsculas/minúsculas e acentos (Equivalente ao ILIKE / LIKE)
    let filtro = #Predicate<Tarefa> { tarefa in
        tarefa.titulo.localizedStandardContains(textoBusca) ||
        tarefa.descricao.localizedStandardContains(textoBusca)
    }

    let descriptor = FetchDescriptor<Tarefa>(predicate: filtro)
    tarefas = (try? modelContext.fetch(descriptor)) ?? []
}
```

4. WHERE com Datas (SELECT * FROM tarefa WHERE data >= ?)

```
func buscarTarefasFuturas() {
    let agora = Date.now

    let filtro = #Predicate<Tarefa> { tarefa in
        tarefa.data >= agora
    }

    let descriptor = FetchDescriptor<Tarefa>(predicate: filtro)
    tarefas = (try? modelContext.fetch(descriptor)) ?? []
}
```

5. Consulta Completa: WHERE + LIKE + ORDER BY + LIMIT

```
func buscaAvancada(texto: String, dataMinima: Date) {
    // Filtro combinado (AND e OR)
    let filtro = #Predicate<Tarefa> { tarefa in
        tarefa.titulo.localizedStandardContains(texto) &&
        tarefa.data >= dataMinima
    }

    // Ordenação
    let ordenacao = [SortDescriptor(\Tarefa.data, order: .forward)]

    // Monta o descritor unindo filtro e ordenação
    var descriptor = FetchDescriptor<Tarefa>(
        predicate: filtro,
        sortBy: ordenacao
    )

    // LIMIT (SELECT ... LIMIT 10)
    descriptor.fetchLimit = 10

    tarefas = (try? modelContext.fetch(descriptor)) ?? []
}
```

Como implementar uma chave primária:

```
// Importações
import Foundation
import SwiftData

// Modelo
@Model
class Tarefa {

    // Chave Primária (PRIMARY KEY UNIQUE)
    @Attribute(.unique) var id: UUID

    // Atributos
    var titulo: String
    var descricao: String
    var data: Date

    // Construtor / Inicializador
    init(id: UUID = UUID(), titulo: String, descricao: String, data: Date = .now) {
        self.id = id
        self.titulo = titulo
        self.descricao = descricao
        self.data = data
    }
}
```

Como implementar uma chave estrangeira:

```
// Importações
import Foundation
import SwiftData

// Modelo Pai: Usuario (1)
@Model
class Usuario {

    // Chave Primária
    @Attribute(.unique) var id: UUID
    var nome: String
    var email: String

    // Relacionamento 1:N (Um usuário possui várias tarefas)
      // deleteRule: .cascade -> Se o usuário for excluído, suas tarefas também serão apagadas
     // inverse: \Tarefa.usuario -> Informa qual propriedade na classe Tarefa aponta de volta para Usuario
    @Relationship(deleteRule: .cascade, inverse: \Tarefa.usuario)
    var tarefas: [Tarefa] = []

    init(id: UUID = UUID(), nome: String, email: String) {
        self.id = id
        self.nome = nome
        self.email = email
    }
}
```

```
// Modelo Filho: Tarefa (N)
@Model
class Tarefa {

    // Chave Primária
    @Attribute(.unique) var id: UUID
    var titulo: String
    var descricao: String
    var data: Date

    // Chave Estrangeira (Referência direta ao objeto Usuario)
    var usuario: Usuario?

     init(id: UUID = UUID(), titulo: String, descricao: String, data: Date = .now, usuario:
Usuario? = nil) {
        self.id = id
        self.titulo = titulo
        self.descricao = descricao
        self.data = data
        self.usuario = usuario
    }
}
```

Como implementar um inner join:

```
func listarTarefas(doUsuario usuario: Usuario) {
        let usuarioId = usuario.id

        // Filtramos comparando o ID do relacionamento
        let filtro = #Predicate<Tarefa> { tarefa in
            tarefa.usuario?.id == usuarioId
        }

        let descriptor = FetchDescriptor<Tarefa>(predicate: filtro)
        tarefas = (try? modelContext.fetch(descriptor)) ?? []
}
```